UỶ BAN NHÂN DÂN THÀNH PHỐ
HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO LAB03 – TÌM KIẾM CỤC BỘ

MÔN: TRÍ TUỆ NHÂN TẠO NÂNG CAO

Giảng viên hướng dẫn: Đỗ Như Tài

Sinh viên thực hiện:
Huỳnh Tấn Phi Hùng - 3124720026
Lê Tấn Phát - 3124720050
Lê Ngọc Phúc – 3124720053
Đỗ Xuân Triệu - 3124720072

TP. HỒ CHÍ MINH, ngày 16 tháng 10 năm 2026

Mục lục

1.Giới thiệu và phương pháp thực nghiệm 1
1.1. Mục tiêu 1
1.2. Môi trường và các tệp mã nguồn 1
1.3. Quy ước thực nghiệm 2
2. Bài 1 – Simple Hill Climbing trên mê cung 2
2.1. Giới thiệu bài toán 2
2.2. Kết quả 2
2.3. Thay đổi thứ tự xét hướng và ngẫu nhiên hóa 3
2.4. Random restart 4
3. Bài 2 – Mô hình hóa bài toán n quân hậu 4
3.1. Giới thiệu bài toán 4
3.2. Mô hình hóa bài toán 4
3.3. Khảo sát các hàm trong notebook 5
3.3.1. Hàm random_board(n) 5
3.3.2. Hàm show_board(board) 5
3.3.3. Hàm conflicts(board) 5
3.4. Phép di chuyển cục bộ và số trạng thái láng giềng 6
3.5. Thực nghiệm minh họa với n=4 6
3.6. Nhận xét và kết luận 6
4. Bài 3 – Steepest-ascent và Stochastic Hill Climbing 7
4.1. Steepest-ascent Hill Climbing (Task 1) 7
4.2. Inefficient Stochastic Hill Climbing (Task 2) 8
4.3. Efficient Stochastic (First-choice) Hill Climbing (Task 3) 8
5. Bài 4 – Random Restart và Simulated Annealing 9
5.1. Hill Climbing với Random Restart (Task 4) 9
5.2. Simulated Annealing (Task 5) 10
6. Bài 5 - Khảo sát lịch giảm nhiệt độ 11
6.1. Bốn lịch giảm nhiệt 11
6.2. Ảnh hưởng của lịch tới chất lượng lời giải SA (n = 8) 11
7. Bài 6 - Tối ưu hàm đếm xung đột bằng Numba 12
7.1. Vì sao đếm theo hàng/đường chéo? 13
7.2. Kết quả đo thời gian 13
7.3. Vì sao phải loại thời gian biên dịch lần đầu? 14
7.4. Cải tiến phép đo thời gian (đề xuất của nhóm) 14
8. Bài 7 – Tìm kiếm cục bộ cho bài toán người du lịch (TSP) 15
8.1. Mô hình hóa 15
8.2. Kết quả so sánh 16
8.3. Toán tử hoán đổi so với 2-opt (n = 30) 18
8.4. Thuật toán di truyền (mở rộng) 18
9. Bài 8 – Tổng hợp thí nghiệm và phân tích 18
9.1. Bảng kết quả tổng hợp 18
9.2. Hành vi hội tụ 20
9.3. Khả năng mở rộng theo kích thước bàn cờ 20
9.4. Các toán tử di chuyển khác (phần nâng cao) 21
10. Nhận xét tổng hợp và Đánh giá 23
10.1. Hiện tượng bế tắc tại cực trị cục bộ (*Local Optimum*) 23

10.2. Hiệu năng của chiến lược khởi động lại ngẫu nhiên (*Random Restart*) 23
10.3. Đánh giá chiến lược luyện kim mô phỏng (*Simulated Annealing* - SA) 23
10.4. Thời gian thực thi và Khả năng mở rộng quy mô 24
10.5. Hạn chế và Khoảng trống của thực nghiệm 24
11. Câu hỏi thảo luận: 24
12. Kết luận và Hướng phát triển 28
12.1. Đánh giá tổng quan về thuật toán Tìm kiếm cục bộ 28
12.2. Hướng phát triển 28
Bảng phân công: 29

1. Giới thiệu và phương pháp thực nghiệm

1.1. Mục tiêu

Bài thực hành khảo sát nhóm thuật toán **tìm kiếm cục bộ** (local search): thay vì duy trì cây tìm kiếm và đường đi như BFS hay A∗, thuật toán chỉ giữ *một trạng thái hiện tại*, đánh giá nó bằng một **hàm mục tiêu** và liên tục di chuyển sang các **trạng thái lân cận** tốt hơn. Cách làm này tốn rất ít bộ nhớ và phù hợp với các bài toán tối ưu mà ta chỉ quan tâm đến lời giải cuối cùng (n quân hậu, người du lịch TSP), nhưng đổi lại có thể bị mắc kẹt ở **cực trị cục bộ** (local optimum), vùng bằng phẳng (plateau) hoặc sống núi (ridge).

Báo cáo trình bày kết quả của tám bài thực hành: (1) Simple Hill Climbing trên mê cung; (2) mô hình hóa bài toán n quân hậu; (3) Steepest-ascent và Stochastic Hill Climbing; (4) Random Restart và Simulated Annealing; (5) lịch giảm nhiệt độ; (6) tối ưu hàm đếm xung đột bằng Numba; (7) tìm kiếm cục bộ cho bài toán người du lịch (TSP); (8) tổng hợp thí nghiệm và phân tích.

1.2. Môi trường và các tệp mã nguồn

Toàn bộ mã được viết bằng Python 3 trong các Jupyter Notebook, dùng các thư viện numpy, pandas, scipy, matplotlib và numba (chỉ cho notebook Bài 6). Mọi thí nghiệm có yếu tố ngẫu nhiên đều đặt **seed cố định** nên có thể chạy lại và cho đúng kết quả về mặt ngẫu nhiên (thời gian chạy phụ thuộc cấu hình máy nên có thể sai khác).

Tệp	Nội dung	Bài
Maze_Example_LocalSearch.ipynb	Simple Hill Climbing trên mê cung chữ L, khoảng cách Manhattan; phân tích cực trị cục bộ	1
n_queens.ipynb	Mô hình n quân hậu; Steepest-ascent, Stochastic HC, First-choice HC, Random Restart, Simulated Annealing; phân tích hành vi, khả năng mở rộng; toán tử di chuyển khác	2, 3, 4, 5, 8
Simulated_Annealing_Schedules.ipynb	Bốn lịch giảm nhiệt (mũ, tuyến tính, fast, logarithmic) và khảo sát tham số	5
n_queens_fast_conflict_calculation_with_numba.ipynb	So sánh hàm conflicts dạng naive $O(n^2)$, tuyến tính $O(n)$ và biên dịch JIT bằng Numba	6
traveling_salesman_problem.ipynb	TSP: Steepest-ascent, Random Restart, Stochastic HC, First-choice HC, SA; so sánh toán tử hoán đổi với 2-opt; thuật toán di truyền (mở rộng)	7
L_maze.txt, maze_helper.py	Dữ liệu và hàm hỗ trợ mê cung (do giảng viên cung cấp)	1

Bảng 1. Danh sách tệp mã nguồn nộp kèm

1.3. Quy ước thực nghiệm

- Với n quân hậu: trạng thái là vector board độ dài n (mỗi cột một quân hậu), hàm mục tiêu f = số cặp quân hậu tấn công nhau, cần cực tiểu hóa; lời giải hợp lệ khi f = 0.
- Với TSP: trạng thái là hoán vị các thành phố, hàm mục tiêu là độ dài tour khép kín, cần cực tiểu hóa. Các thành phố được sinh ngẫu nhiên trong hình vuông đơn vị.
- Mỗi cấu hình của n quân hậu được chạy **100 lần** trên các bàn cờ ngẫu nhiên khác nhau; ở bài khả năng mở rộng dùng 30 lần cho mỗi kích thước. Thời gian đo bằng time.perf_counter().
- Tỉ lệ thành công của n quân hậu là phần trăm lần chạy kết thúc với f = 0; với TSP so sánh độ dài tour và mức cải thiện tương đối so với tour ban đầu.

2. Bài 1 – Simple Hill Climbing trên mê cung

2.1. Giới thiệu bài toán

Bài toán yêu cầu giải quyết bài toán tìm đường đi từ điểm xuất phát S đến điểm đích G trên một bản đồ mê cung dạng lưới kích thước 15×15 (L_maze.txt) sử dụng thuật toán tìm kiếm cục bộ Leo đồi (Hill Climbing).

- **Không gian trạng thái và Hàm sinh lân cận:** Mỗi trạng thái được biểu diễn bằng tọa độ (r, c) của tác nhân trên lưới. Hàm expand(maze, pos) có nhiệm vụ kiểm tra 4 hướng di chuyển cơ bản gồm Bắc (N), Đông (E), Nam (S), Tây (W) và loại bỏ các ô bị chặn bởi tường ('X'), trả về danh sách các hướng và tọa độ lân cận hợp lệ.
- **Hàm mục tiêu (Heuristic):** Thuật toán sử dụng khoảng cách Manhattan giữa vị trí hiện tại $n = (r_n, c_n)$ và đích đến $G = (r_G, c_G)$ làm hàm đánh giá cần cực tiểu hóa:

$$h(n) = |r_n - r_G| + |c_n - c_G|$$

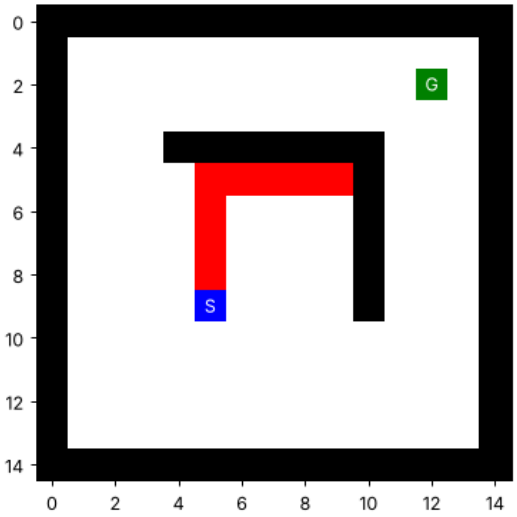
(càng nhỏ càng tốt, h = 0 tại G).

2.2. Kết quả

Khi chạy thuật toán local_search với cấu hình ưu tiên hướng mặc định (ORDER = "NESW", RANDOMIZATION = False), tác nhân bắt đầu xuất phát từ $S(9,5)$ với $h(S) = 14$. Quá trình tìm kiếm diễn ra qua 8 bước lặp trong thời gian khoảng 6.77 ms.

Chỉ số	Giá trị
Ô xuất phát S / ô đích G	(9, 5) / (2, 12)
h(S)	14
Vị trí dừng	(5, 9) sau 8 bước
h tại vị trí dừng	6
Đã tới đích G?	Không

Bảng 2. Kết quả chạy Simple Hill Climbing trên mê cung (thứ tự hướng NESW)



Hình 1. Đường đi của hill climbing (màu đỏ): từ S đi lên rồi sang phải và dừng sát bức tường

Quan sát quỹ đạo thực nghiệm, tác nhân bắt đầu tại ô (9,5) (h=14) sau đó di chuyển lên phía Bắc 4 ô đến $(5,5)$, sau đó rẽ phải sang Đông 4 ô dọc theo cạnh ngang của bức tường chữ L để tiến đến ô $(5,9)$ ($h = 6$). Tại ô này, hai hướng giúp tiến gần đích hơn (đi lên Bắc và sang phải Đông) đều bị chắn bởi các ô tường 'X'. Trong khi đó, hai hướng láng giềng hợp lệ còn lại là đi xuống Nam hoặc sang trái Tây đều làm tăng khoảng cách Manhattan lên 7.

Vì không có láng giềng nào tốt hơn trạng thái hiện tại, thuật toán lập tức dừng lại theo điều kiện dừng cục bộ. Đây là hiện tượng **mắc kẹt tại cực trị cục bộ (local optimum)**: ô $(5,9)$ có giá trị tối ưu so với mọi ô lân cận trực tiếp của nó nhưng thực chất vẫn cách đích G một khoảng $h = 6$. Do tính chất tham lam và chỉ chấp nhận bước cải thiện nghiêm ngặt, Hill Climbing không thể tự tìm đường vòng qua các đầu mút của bức tường chữ L để tiếp cận đích.

2.3. Thay đổi thứ tự xét hướng và ngẫu nhiên hóa

Để kiểm tra tính ổn định của điểm dừng, thực nghiệm được mở rộng bằng cách chạy thử toàn bộ 24 hoán vị cố định của thứ tự hướng (ORDER) và 200 lần chạy với cơ chế chọn hướng ngẫu nhiên ở mỗi bước (RANDOMIZATION = True).

- Ta thử toàn bộ 4! = 24 cấu hình hoán vị và 200 lần chạy ngẫu nhiên ở mỗi bước (random=True) **đều dừng chính xác tại cùng một ô $(5, 9)$** và không có lần nào thành công tới đích G.
- Việc thay đổi thứ tự xét hướng hay trộn ngẫu nhiên chỉ đóng vai trò phân định lựa chọn khi có các ô láng giềng đồng hạng (cùng giá trị h). Do đó, các yếu tố này chỉ làm thay đổi **hình dạng hình học của đường đi** (ví dụ: đi theo đường bậc thang zig-zag thay vì đi thẳng rồi rẽ), chứ không làm thay đổi **điểm dừng cuối cùng**. Nguyên nhân sâu xa là toàn bộ không gian lân cận hướng về phía trước đều bị giới hạn bởi cấu trúc hình học của bức tường chữ L, ép mọi đường đi giảm đơn điệu h phải hội tụ về cùng một điểm tại $(5,9)$.

2.4. Random restart

Để giải quyết bài toán triệt để hơn mà không bị giới hạn bởi điểm xuất phát cố định, phương pháp **Random Restart** đã được khảo sát bằng cách cho thuật toán khởi tạo độc lập từ toàn bộ 156 ô trống khác nhau trên mê cung.

- Thuật toán thành công tìm đến được đích G từ 102 **ô (chiếm tỉ lệ** 65%**)**, và tiếp tục bị kẹt ở cực trị cục bộ từ 54 **ô còn lại (chiếm** 35%**)**.
- *Random Restart* là một cải tiến đơn giản nhưng hiệu quả để vượt qua hạn chế của cực trị cục bộ, biến một thuật toán không đầy đủ thành một thuật toán có xác suất tìm thấy nghiệm cao. Tuy nhiên, phương pháp này vẫn mang tính xác suất và không bảo đảm thành công tuyệt đối trong mọi trường hợp, bởi kết quả phụ thuộc hoàn toàn vào việc điểm khởi tạo ngẫu nhiên có nằm trong "vùng hút" (basin of attraction) dẫn thẳng tới đích toàn cục hay không.

3. Bài 2 – Mô hình hóa bài toán n quân hậu

3.1. Giới thiệu bài toán

Bài toán n quân hậu yêu cầu đặt n quân hậu lên bàn cờ n×n sao cho không có hai quân hậu nào cùng hàng, cùng cột hoặc cùng đường chéo. Trong bài thực hành này, không gian trạng thái được thu hẹp bằng cách chỉ xét các bàn cờ có **đúng một quân hậu trên mỗi cột**. Vì vậy, điều kiện không xung đột cột luôn được thỏa mãn và chỉ cần kiểm tra xung đột theo hàng và đường chéo.

3.2. Mô hình hóa bài toán

Trạng thái được biểu diễn bằng vector $q = (q_1, q_2, \ldots, q_n)$, trong đó $q_i \in \{0, 1, \ldots, n-1\}$ là chỉ số hàng của quân hậu ở cột i. Mỗi phần tử tương ứng với một cột, do đó mỗi cột luôn có đúng một quân hậu.

Thành phần	Mô tả
Trạng thái	Vector $q = (q_1, q_2, \ldots, q_n)$, với q_i là chỉ số hàng của quân hậu ở cột i.
Không gian trạng thái	Có n lựa chọn hàng cho mỗi cột, nên tổng số trạng thái là n^n. Với n=4: 4^4=256; với n=8: 8^8=16.777.216.
Hàm mục tiêu	f(q) = conflicts(q), là số cặp quân hậu tấn công nhau theo hàng hoặc đường chéo. Mục tiêu là tối thiểu hóa f(q).
Phép di chuyển cục bộ	Chọn một cột và chuyển quân hậu ở cột đó sang một hàng khác, giữ nguyên vị trí các quân hậu còn lại.
Số trạng thái láng giềng	Mỗi cột có n−1 hàng thay thế. Tổng số láng giềng khác trạng thái hiện tại là n(n−1): 12 khi n=4 và 56 khi n=8.

Điều kiện đạt nghiệm	f(q)=0, nghĩa là không có cặp quân hậu nào tấn công nhau.
Điều kiện dừng	Đạt f(q)=0 hoặc không còn bước đi giúp giảm f(q). Trường hợp sau có thể là cực trị cục bộ, chưa chắc là nghiệm.

Bảng 3. Mô hình hóa bài toán n quân hậu cho tìm kiếm cục bộ.

3.3. Khảo sát các hàm trong notebook

3.3.1. Hàm random_board(n)

Hàm random_board(n) tạo một vector ngẫu nhiên gồm n phần tử. Mỗi phần tử là chỉ số hàng của quân hậu tại một cột, được chọn trong khoảng từ 0 đến n−1. Trạng thái sinh ra luôn có đúng một quân hậu trên mỗi cột, nhưng vẫn có thể có xung đột theo hàng hoặc đường chéo.

```
def random_board(n):
    """Creates a random board of size n x n. Note that only a single queen is placed in each column!"""

    return(np.random.randint(0,n, size = n))
```

3.3.2. Hàm show_board(board)

Hàm show_board(board) trực quan hóa vector trạng thái thành bàn cờ n×n, hiển thị vị trí các quân hậu và hỗ trợ quan sát cấu hình. Hàm này phục vụ mục đích hiển thị, không làm thay đổi trạng thái.

3.3.3. Hàm conflicts(board)

Hàm conflicts(board) tính số cặp quân hậu đang tấn công nhau. Với hai quân hậu ở cột i và j, chúng xung đột nếu $q_i=q_j$ (cùng hàng) hoặc $|q_i−q_j|=|i−j|$ (cùng đường chéo).

Notebook đếm số quân hậu trên từng hàng và hai nhóm đường chéo. Nếu một hàng hoặc đường chéo chứa k quân hậu, số cặp xung đột trong nhóm đó là C(k,2)=k(k−1)/2. Cộng số cặp của tất cả các nhóm sẽ thu được tổng số xung đột.

```
def comb2(n): return n*(n-1)//2 # this is n choose 2 equivalent to math.comb(n, 2); // is int division

def conflicts(board):
    """Calculate the number of conflicts, i.e., the objective function."""

    n = len(board)

    horizontal_cnt = [0] * n
    diagonal1_cnt = [0] * 2 * n
    diagonal2_cnt = [0] * 2 * n

    for i in range(n):
        horizontal_cnt[board[i]] += 1
        diagonal1_cnt[i + board[i]] += 1
        diagonal2_cnt[i - board[i] + n] += 1

    return sum(map(comb2, horizontal_cnt + diagonal1_cnt + diagonal2_cnt))
```

Do phương pháp sử dụng các mảng đếm và chỉ duyệt qua n cột, độ phức tạp thời gian của hàm là O(n). Cách làm này tránh phải kiểm tra trực tiếp mọi cặp quân hậu.

3.4. Phép di chuyển cục bộ và số trạng thái láng giềng

Từ một trạng thái hiện tại, ta chọn một cột bất kỳ và chuyển quân hậu tại cột đó sang một hàng khác. Vì có n cột và mỗi cột có n−1 hàng thay thế, số trạng thái láng giềng khác trạng thái hiện tại là:

N_láng_giềng = n(n − 1)

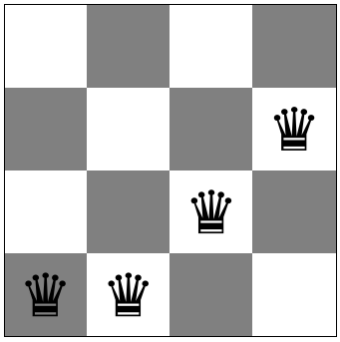
Kích thước bàn cờ	Số cột	Hàng thay thế mỗi cột	Số láng giềng
4×4	4	3	12
8×8	8	7	56

Số láng giềng là số trạng thái có thể tạo ra từ một trạng thái hiện tại khi thực hiện một phép di chuyển. Nếu thuật toán quét toàn bộ lân cận, nó cần đánh giá các trạng thái này; nếu chỉ sinh lân cận ngẫu nhiên, số trạng thái đánh giá trong mỗi lần thử có thể ít hơn.

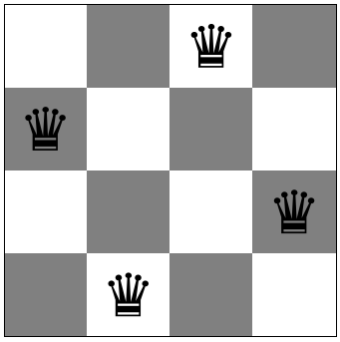
3.5. Thực nghiệm minh họa với n=4

Theo output đã lưu trong notebook với seed 1234, bàn cờ ngẫu nhiên là vector [3, 3, 2, 1], có f(q)=4 xung đột. Cấu hình [1, 3, 0, 2] có f(q)=0, do đó là một nghiệm hợp lệ.

Trường hợp	Vector trạng thái	f(q)	Kết luận
Bàn cờ ngẫu nhiên	[3, 3, 2, 1]	4	Chưa hợp lệ
Cấu hình cho trước	[1, 3, 0, 2]	0	Nghiệm hợp lệ



Hình 2a. Bàn cờ 4×4 ngẫu nhiên [3, 3, 2, 1], f(q)= 4



Hình 2b. Bàn cờ [1, 3, 0, 2], f(q)= 0, là nghiệm hợp lệ.

3.6. Nhận xét và kết luận

Biểu diễn bằng vector giúp giảm số biến cần quản lý so với ma trận n×n. Ràng buộc mỗi cột có đúng một quân hậu được thỏa mãn ngay từ đầu, do đó quá trình đánh giá chỉ cần tập trung vào xung đột hàng và đường chéo.

Hàm conflicts(board) định lượng mức độ vi phạm ràng buộc: giá trị càng thấp thì trạng thái càng gần với một nghiệm hợp lệ. Tuy nhiên, việc giảm xung đột qua từng

bước không bảo đảm thuật toán luôn tìm được nghiệm, vì phương pháp leo đồi có thể dừng tại cực trị cục bộ khi không còn bước cải thiện.

Khi n tăng, không gian trạng thái tăng theo n^n, trong khi số láng giềng của mỗi trạng thái tăng theo $n(n-1)$. Vì vậy, cần lựa chọn chiến lược tìm kiếm phù hợp để cân bằng giữa chất lượng lời giải và chi phí tính toán. Các phương pháp như Random Restart và Simulated Annealing sẽ được đánh giá trong phần thực nghiệm.

4. Bài 3 – Steepest-ascent và Stochastic Hill Climbing

4.1. Steepest-ascent Hill Climbing (Task 1)

Cơ chế vận hành: Ở mỗi bước đi, thuật toán thực hiện vét cạn và đánh giá toàn bộ không gian lân cận có kích thước $n(n-1)$ (thông qua thao tác dịch chuyển từng quân hậu sang các vị trí hàng khác trong cùng cột). Sau đó, nó tham lam chọn ra trạng thái mang lại mức cải thiện tối ưu nhất — tức là cấu hình làm giảm giá trị hàm mục tiêu $f(\text{board})$ mạnh nhất.

Chính sách phá hòa (*Tie-breaking*): Khi xuất hiện nhiều nước đi lân cận cùng đạt mức đánh giá tốt nhất (đồng hạng f nhỏ nhất), thuật toán sẽ tiến hành **lựa chọn ngẫu nhiên** thay vì bám theo thứ tự duyệt mảng cứng nhắc, giúp triệt tiêu hiện tượng thiên lệch hướng và tăng tính khách quan cho đường đi.

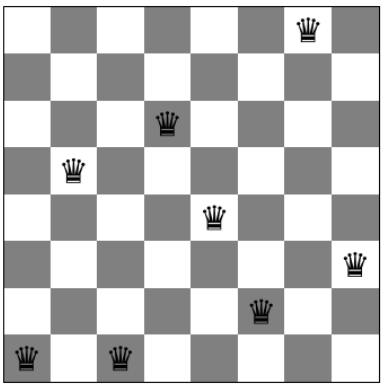
Điều kiện dừng linh hoạt: Quá trình tìm kiếm sẽ kết thúc khi thỏa mãn một trong hai kịch bản:

1. Tìm thấy lời giải tối ưu toàn cục hoàn toàn không còn xung đột ($f = 0$).
2. Thuật toán cạn kiệt đường tiến (không còn lân cận nào cải thiện được giá trị f), chính thức rơi vào bẫy cực trị cục bộ (*local optimum*).

Thông tin thu thập: Mỗi lượt thực thi ghi nhận đầy đủ các thông số: cấu hình bàn cờ đích, giá trị f cuối cùng, tổng số bước lặp, chi phí tính toán (số lần gọi hàm conflicts()) và toàn bộ lịch sử biến động của f.

Minh họa thực nghiệm trên bàn cờ 8×8 (với Seed: 1234):

- **Cấu hình khởi tạo:** [7, 3, 6, 5, 4, 4, 0, 1] ghi nhận số cặp quân hậu tấn công nhau ban đầu là $f = 9$.
- **Diễn biến & Kết quả:** Sau vỏn vẹn 4 bước dịch chuyển cùng tổng cộng **280 lượt đánh giá hàm mục tiêu**, thuật toán khựng lại và dừng hẳn tại cấu hình [7, 3, 7, 2, 4, 6, 0, 5] với $f = 1$ (như minh họa tại **Hình 3**).
- **Phân tích chuyên sâu:** Việc dừng chân ở $f = 1$ thay vì chạm đích 0 minh chứng rõ nét cho nhược điểm chí mạng của phương pháp leo đồi tham lam: do chỉ nhìn thấy các bước đi lân cận cục bộ, thuật toán đã bất lực trước một cực trị cục bộ (hoặc vùng sườn phẳng), nơi mọi bước di chuyển tiếp theo đều làm tăng hoặc giữ nguyên số xung đột.



Hình 3. Kết quả steepest-ascent trên bàn cờ 8×8: kẹt ở cực trị cục bộ với f = 1

4.2. Inefficient Stochastic Hill Climbing (Task 2)

Điểm khác biệt cốt lõi: Thay vì tham lam chọn lọc khắt khe trạng thái có mức tối ưu tốt nhất như phương pháp *Steepest-ascent*, biến thể ngẫu nhiên không hiệu quả cho phép thuật toán **chọn ngẫu nhiên một nước đi bất kỳ** nằm trong tập hợp các lân cận có tính cải thiện (với điều kiện giá trị hàm mục tiêu f giảm ngặt so với hiện tại).

Đánh giá ưu - nhược điểm:

- **Ưu điểm (*Đa dạng hóa đường đi*):** Việc không giới hạn ở một nước đi tối ưu duy nhất giúp mở rộng không gian tìm kiếm, tạo sự linh hoạt lớn hơn trên quỹ đạo di chuyển và có khả năng né tránh một số ngõ cụt cục bộ đơn giản.
- **Hạn chế (*Chi phí tính toán cao*):** Tương tự như Steepest-ascent, thuật toán vẫn buộc phải quét và đánh giá toàn bộ không gian lân cận có kích thước $n(n-1)$ ở mỗi bước lặp. Chính vì sự lãng phí tài nguyên khi phải duyệt toàn bộ nhưng lại chọn một cách ngẫu nhiên (không tận dụng triệt để giá trị tốt nhất), phương pháp này được định danh là **"không hiệu quả"** (*inefficient*).

4.3. Efficient Stochastic (First-choice) Hill Climbing (Task 3)

Chiến lược tối ưu hóa lượt sinh lân cận: Khác biệt hoàn toàn với hai biến thể trước, thuật toán không tiến hành quét toàn bộ không gian xung quanh. Thay vào đó, tại mỗi bước, nó **sinh ngẫu nhiên duy nhất một trạng thái lân cận** (bằng cách chọn ngẫu nhiên một cột bất kỳ và thay đổi vị trí hàng của quân hậu sang một hàng mới khác với hàng hiện tại) và tiến hành chấp nhận ngay lập tức nếu giá trị mục tiêu f giảm ngặt.

Bài toán tiêu chí dừng (*Dừng khi không cải thiện*): Do cơ chế không duyệt toàn bộ lân cận, thuật toán đòi hỏi một mốc giới hạn để xác định thời điểm bế tắc. Tiêu chí được thiết lập là: **dừng lại sau x lần thử liên tiếp không tìm thấy trạng thái nào cải thiện hơn**.

Thiết lập mô hình toán học dựa trên bài toán *Coupon Collector*:

- Gọi $N = n(n-1)$ là tổng số lượng trạng thái lân cận khả dĩ.
- Ứng dụng mô hình lấy mẫu có hoàn lại tương tự như bài toán sưu tập phiếu thưởng (*coupon collector problem*), sau tổng cộng $N(\ln N + c)$ lần thử, xác suất thuật toán đã vét cạn toàn bộ N lân cận xấp xỉ bằng $\exp(-e^{-c})$.
- Lựa chọn hằng số $c = 2$ cho phép đạt độ tin cậy xấp xỉ 87%, từ đó thiết lập công thức tính ngưỡng dừng tối ưu:

$$x = \lceil N(\ln N + 2) \rceil, \quad \text{với } N = n(n-1)$$

Khảo sát giá trị thực nghiệm theo kích thước bàn cờ (n):

- **Trường hợp $n = 4$:** Tổng số lân cận $N = 12 \Longrightarrow$ Ngưỡng dừng tương ứng $x = 54$.
- **Trường hợp $n = 8$:** Tổng số lân cận $N = 56 \Longrightarrow$ Ngưỡng dừng tương ứng $x = 338$.
- **Nhận định quy mô:** Ngưỡng giới hạn x tăng trưởng theo bậc xấp xỉ $O(n^2 \ln n)$ khi kích thước bàn cờ mở rộng, giúp cân bằng hoàn hảo giữa tốc độ tính toán nhanh chóng và độ bao phủ của không gian tìm kiếm.

5. Bài 4 – Random Restart và Simulated Annealing

5.1. Hill Climbing với Random Restart (Task 4)

Thiết lập quy trình thực nghiệm:

- Áp dụng chiến lược khởi động ngẫu nhiên lại từ đầu (*Random Restart*) nhằm khắc phục hiện tượng bế tắc tại cực trị cục bộ cho bộ ba biến thể leo đồi.
- Mỗi thuật toán được cấu hình chạy lặp lại tối đa **100 lần** xuất phát từ các cấu hình bàn cờ ngẫu nhiên mới, thiết lập cơ chế lưu trữ kết quả tối ưu và **dừng sớm (*early stopping*)** ngay khi tìm thấy lời giải không xung đột hoàn toàn ($f = 0$).

Tổng hợp kết quả thực nghiệm (*Bảng 4*): với 100 lần khởi động, cả ba biến thể đều giải được 100% các lần thử trên n = 4 và n = 8.

Thuật toán + restart	n	% giải được (≤ 100 lần)	TB số lần khởi động	Max số lần khởi động
Steepest-ascent	4	100	2,32	15
Steepest-ascent	8	100	7,90	28
Stochastic HC (mọi nước đi)	4	100	2,89	14
Stochastic HC (mọi nước đi)	8	100	6,94	45
First-choice HC	4	100	2,65	12
First-choice HC	8	100	8,69	51

Bảng 4. Hill climbing với random restart (100 lần thử mỗi dòng)

Luận giải lý thuyết xác suất và kỳ vọng khởi động:

- Giả định xác suất thành công trong một lượt thực thi đơn là p, kỳ vọng số lần khởi động cần thiết để chạm tới lời giải tối ưu sẽ tuân theo phân phối hình học với giá trị trung bình xấp xỉ $\frac{1}{p}$.
- Tại quy mô bàn cờ $n = 8$, khi xác suất thành công của mỗi lượt chạy đơn chỉ duy trì ở mức $8\% - 19\%$ (đối chiếu với *Bảng 11*), số lần khởi động trung bình thực tế chỉ dao động trong khoảng 7 **đến** 9 **lần**.

- **Nhận định chung:** Hạn mức cấu hình lên đến 100 lần khởi động là một sự chuẩn bị rất an toàn và dư dả. Việc gia tăng thêm số lần restart chỉ thực sự phát huy tác dụng khi xác suất thành công đơn lẻ p suy giảm xuống mức cực kỳ nhỏ.

5.2. Simulated Annealing (Task 5)

Bản chất và cơ chế cốt lõi:

- Thuật toán Luyện kim mô phỏng (*Simulated Annealing* - SA) được phát triển như một biến thể tối ưu của *Stochastic Hill Climbing*.
- Điểm đột phá của SA nằm ở việc trao cho thuật toán khả năng **chấp nhận các bước đi xấu hơn** (tức là trạng thái có giá trị hàm mục tiêu cao hơn trạng thái hiện tại) thông qua một xác suất biến thiên theo thời gian.

Quy trình đánh giá và chuyển trạng thái tại mỗi vòng lặp t:

- Thuật toán tiến hành sinh ngẫu nhiên một trạng thái lân cận, sau đó tính toán biên độ chênh lệch năng lượng:

$$\Delta = f(\text{mới}) - f(\text{hiện tại})$$

- **Chính sách chấp nhận trạng thái:**
  - Nếu $\Delta < 0$ (cải thiện nghiệm): **Chấp nhận tuyệt đối** trạng thái mới.
  - Nếu $\Delta \geq 0$ (làm xấu đi hoặc giữ nguyên): **Chấp nhận với xác suất ngẫu nhiên** tuân theo hàm phân phối Boltzmann:

$$p = \exp\left(-\frac{\Delta}{T}\right)$$

Vai trò điều phối của nhiệt độ T và lịch làm nguội:

- Nhiệt độ T suy giảm dần theo một hàm lịch trình nhiệt $T(t)$ đã thiết lập.
- **Giai đoạn đầu (T cao):** Xác suất chấp nhận bước xấu lớn, giúp thuật toán thực hiện quá trình **khám phá diện rộng** ($exploration$) toàn bộ không gian trạng thái, triệt tiêu nguy cơ bị dính chặt vào bẫy cục bộ.
- **Giai đoạn sau (T thấp):** Xác suất chấp nhận bước xấu thu hẹp dần, thuật toán chuyển sang chế độ **khai thác sâu** ($exploitation$), hành xử tương tự như thuật toán leo đồi truyền thống để hội tụ nhanh về vùng nghiệm tối ưu.

Tiêu chí dừng và cấu trúc kết quả trả về: Quá trình tìm kiếm của Simulated Annealing sẽ chấm dứt khi thỏa mãn một trong các điều kiện sau:

1. Chạm đích lời giải hoàn hảo không có xung đột ($f = 0$).
2. Nhiệt độ hệ thống giảm xuống thấp hơn ngưỡng tối thiểu quy định ($T < T_{\min} = 10^{-3}$).
3. Cạn kiệt ngân sách vòng lặp (vượt quá giới hạn 5000 vòng lặp).

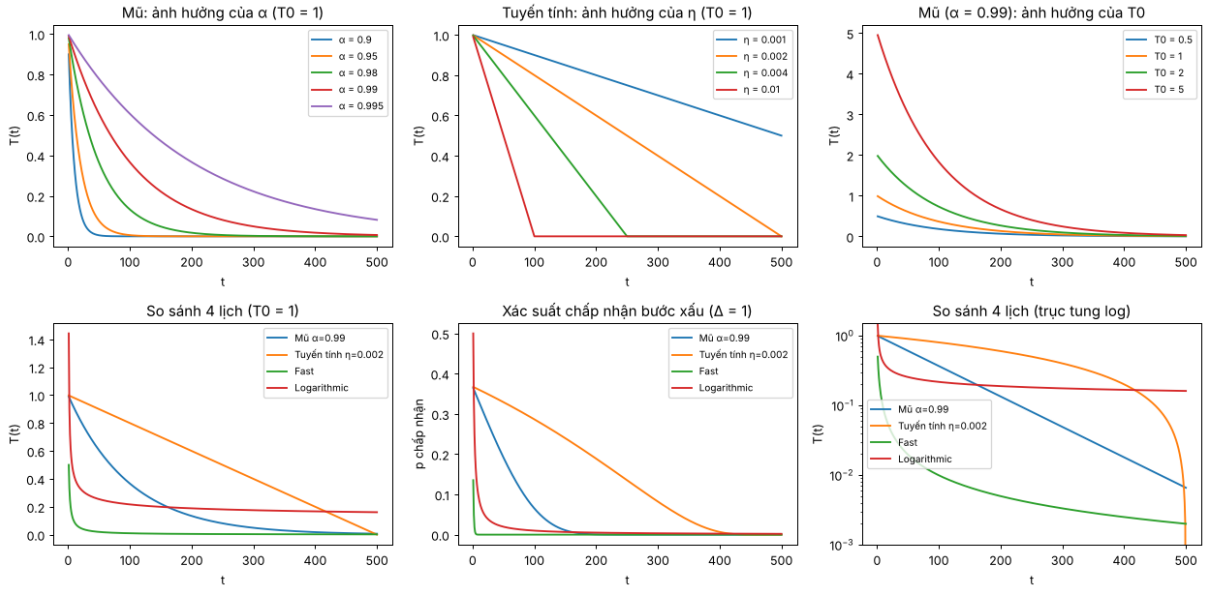
- **Kết quả cuối cùng:** Trạng thái tốt nhất từng được ghi nhận trong toàn bộ tiến trình tìm kiếm sẽ được trích xuất làm phương án tối ưu trả về.

6. Bài 5 - Khảo sát lịch giảm nhiệt độ

6.1. Bốn lịch giảm nhiệt

Lịch	Công thức	Tham số
Mũ (exponential)	$T(t) = T_0 \cdot \alpha^t$	$\alpha \in (0, 1)$
Tuyến tính (linear)	$T(t) = T_0 - \eta \cdot t$	$\eta > 0$
Fast	$T(t) = T_0 / (1 + t)$	không
Logarithmic	$T(t) = T_0 / \ln(1 + t),\ t > 0$	không

Bảng 5. Các lịch giảm nhiệt độ



Hình 4. Ảnh hưởng của α, η, T_0 và so sánh bốn lịch (notebook Simulated_Annealing_Schedules)

Từ Hình 4: (a) α càng gần 1 thì nguội càng chậm, thời gian để T giảm còn 10% là ln 0,1 / ln α vòng lặp (khoảng 230 vòng với α = 0,99 và 2300 vòng với α = 0,999); (b) η lớn làm T về 0 sau T_0/η vòng và dừng đột ngột; (c) T_0 quyết định xác suất chấp nhận bước xấu ngay từ đầu; (d-f) lịch fast giảm rất nhanh lúc đầu rồi còn một "đuôi" rất thấp, logarithmic giảm nhanh lúc đầu rồi gần như đi ngang (còn khoảng 0,16 sau 500 vòng), tuyến tính giữ xác suất chấp nhận cao nhất trong khoảng 400 vòng đầu rồi tụt về 0.

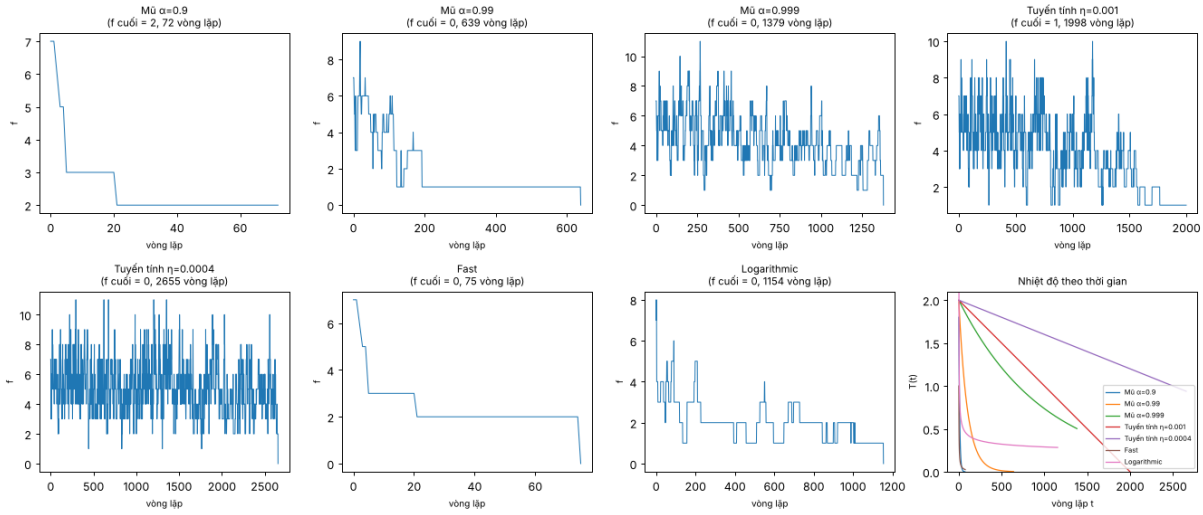
6.2. Ảnh hưởng của lịch tới chất lượng lời giải SA (n = 8)

Với $T_0 = 2$ (cùng bậc với độ chênh Δ thường gặp là 1-3), mỗi lịch được chạy 100 lần trên các bàn cờ ngẫu nhiên 8×8 (seed cố định):

Lịch nhiệt độ	% thành công (f = 0)	TB f cuối	TB số vòng lặp	TB thời gian (ms)
Mũ α=0.9	7	1.63	71	1.0
Mũ α=0.99	60	0.40	530	7.7
Mũ α=0.999	98	0.02	1717	21.6

Lịch nhiệt độ	% thành công (f = 0)	TB f cuối	TB số vòng lặp	TB thời gian (ms)
Tuyến tính η=0.001	67	0.33	1684	30.4
Tuyến tính η=0.0004	93	0.07	3296	52.4
Fast	88	0.12	733	11.3
Logarithmic	100	0.00	712	13.9

Bảng 6. So sánh lịch giảm nhiệt độ trong Simulated Annealing (n = 8, 100 lần chạy)



Hình 5. Một lần chạy SA cho mỗi lịch (cùng bàn cờ ban đầu) và đường nhiệt độ tương ứng

Nhận xét: Nguội quá nhanh (mũ α = 0,9) khiến thuật toán đóng băng sau khoảng 70 vòng lặp và hầu như chỉ còn là hill climbing (7% thành công). Giảm nhiệt chậm hơn (mũ α = 0,999; tuyến tính η = 0,0004) nâng tỉ lệ thành công lên trên 90% nhưng tốn nhiều vòng lặp và thời gian hơn (khoảng 20-50 ms). Lịch logarithmic đạt 100% thành công chỉ với khoảng 712 vòng lặp, và lịch fast đạt 88% với 733 vòng lặp: với bài toán này, những lịch "giảm nhanh lúc đầu rồi giữ nhiệt độ thấp nhưng khác 0 lâu" cho sự cân bằng tốt nhất giữa chất lượng và chi phí. Ta dùng lịch mũ α = 0,999 (T_0 = 2, 98% thành công) làm cấu hình mặc định ở Bài 8.

Chỉ riêng việc đổi lịch nhiệt (cùng T_0 và cùng số vòng lặp tối đa) làm tỉ lệ thành công thay đổi từ 7% đến 100%; điều này cho thấy SA nhạy với việc hiệu chỉnh lịch nhiệt độ và nên được khảo sát bằng thực nghiệm trên từng bài toán.

7. Bài 6 - Tối ưu hàm đếm xung đột bằng Numba

Hàm mục tiêu conflicts() được gọi rất nhiều lần trong mọi thuật toán nên đáng để tối ưu. Notebook so sánh ba cài đặt: conflicts_naive (vòng lặp lồng nhau, $O(n^2)$), conflicts (đếm theo hàng/đường chéo, O(n)) và conflicts_jit (cùng thuật toán O(n) nhưng biên dịch JIT bằng Numba).

7.1. Vì sao đếm theo hàng/đường chéo?

Nếu có k quân hậu trên cùng một hàng (hoặc cùng một đường chéo) thì chúng tạo ra C(k, 2) = k(k-1)/2 cặp xung đột. Do đó không cần so sánh từng cặp quân hậu: chỉ cần đếm số quân hậu trên mỗi hàng, mỗi đường chéo "/" (chỉ số $i + q_i$) và mỗi đường chéo "\" (chỉ số $i - q_i + n$) trong một lượt duyệt n quân hậu, sau đó cộng C(k, 2) của các ô đếm:

f(board) = Σ C(k, 2) (lấy tổng theo mọi hàng và mọi đường chéo, k là số quân hậu trên đó).

Có n hàng và 2n đường chéo mỗi loại nên tổng chi phí là O(n), thay vì phải so sánh C(n, 2) = n(n-1)/2 cặp quân hậu ($O(n^2)$) như cách làm ngây thơ.

7.2. Kết quả đo thời gian

Thời gian trung bình tính hàm mục tiêu cho một bàn cờ (micro giây), đo trên 20 bàn cờ ngẫu nhiên cho mỗi kích thước (seed 1234), bằng đúng mã trong notebook:

Kích thước n	Naive $O(n^2)$ (µs)	Tuyến tính O(n) (µs)	JIT Numba (µs)	Tỉ lệ naive / tuyến tính
8	141,2	33,6	2,17	4,2
50	1656,9	138,3	4,02	12,0
100	4327,3	132,5	4,61	32,7
150	4873,9	160,2	3,97	30,4
200	8482,6	221,9	4,75	38,2
250	12564,3	284,8	5,67	44,1
300	24336,9	326,5	6,45	74,5
350	32857,5	428,6	7,00	76,7
400	31527,7	469,9	10,61	67,1
450	52544,1	652,6	8,35	80,5

Bảng 7. Thời gian tính hàm mục tiêu cho một bàn cờ

Với bàn cờ 8×8, hàm biên dịch conflicts_jit mất 2,17 µs mỗi lần gọi, so với 33,6 µs của cài đặt tuyến tính thuần Python (nhanh hơn khoảng 15 lần) và 141,2 µs của cài đặt ngây thơ. Khi n tăng, lợi thế của JIT càng rõ: ở n = 450, conflicts_jit chỉ mất 8,35 µs, nhanh hơn cách tuyến tính (652,6 µs) khoảng 78 lần và nhanh hơn cách ngây thơ (52 544,1 µs) khoảng 6300 lần. Cùng thuật toán O(n) nhưng chạy bằng mã máy đã biên dịch nên JIT loại bỏ gần hết chi phí thông dịch của Python: lợi ích nằm ở hệ số hằng, không đổi bậc O(n). Với n nhỏ, chênh lệch tuyệt đối chỉ vài chục đến vài trăm µs, nên lợi ích của JIT chỉ thật sự đáng kể khi hàm được gọi rất nhiều lần như trong các thuật toán tìm kiếm cục bộ.

Khi n tăng gấp đôi (n = 200 → 400), thời gian của cách naive tăng khoảng 3,7 lần (gần với 4 lần của $O(n^2)$), cách tuyến tính tăng khoảng 2,1 lần và JIT khoảng 2,2 lần (xấp xỉ O(n)). Ở n = 450 cách đếm tuyến tính nhanh hơn cách ngây thơ khoảng 80 lần. Tuy

nhiên các đường thời gian không hoàn toàn tăng dần: naive ở n = 400 thấp hơn ở n = 350, cách tuyến tính ở n = 100 thấp hơn ở n = 50, JIT ở n = 150 thấp hơn ở n = 100 và ở n = 450 thấp hơn ở n = 400. Về lý thuyết thời gian phải tăng theo n, nên đây là nhiễu đo chứ không phải do thuật toán: mã đo của notebook dùng timeit.timeit(..., number=1), tức mỗi trường hợp chỉ được đo một lần, nên ở thang vài micro giây kết quả dễ bị ảnh hưởng bởi các tác vụ khác của hệ điều hành, bộ nhớ đệm CPU hay bộ thu gom rác của Python. Nhiễu không làm thay đổi kết luận chính nhưng làm các con số cụ thể kém tin cậy; mục 7.4 đề xuất cách khắc phục.

Ghi chú: thời gian tuyệt đối phụ thuộc cấu hình máy. Toàn bộ số liệu trong Bảng 7 lấy từ một lần chạy notebook n_queens_fast_conflict_calculation_with_numba.ipynb (chạy từ đầu đến cuối, dùng nguyên mã đo của notebook gốc), nên ba cột so sánh được trực tiếp với nhau. Đồ thị thời gian theo n của cả ba cài đặt có trong notebook này.

7.3. Vì sao phải loại thời gian biên dịch lần đầu?

Numba biên dịch hàm ở lần gọi đầu tiên (just-in-time compilation). Lần gọi đầu vì vậy bao gồm cả thời gian biên dịch (thường hàng trăm mili giây), lớn hơn nhiều so với thời gian chạy thực của hàm (vài micro giây). Nếu không "làm nóng" hàm bằng một lần gọi trước, thời gian biên dịch bị tính vào phép đo và làm sai lệch việc so sánh tốc độ. Vì thế notebook gọi conflicts_jit(random_board(4)) một lần trước vòng đo. Lợi ích của JIT nằm ở hệ số hằng (giảm chi phí thông dịch của Python), không thay đổi bậc O(n) của thuật toán.

7.4. Cải tiến phép đo thời gian (đề xuất của nhóm)

Để giảm nhiễu đo đã nêu ở mục 7.2, nhóm đề xuất lặp lại mỗi phép đo nhiều lần và lấy giá trị nhỏ nhất: thay timeit.timeit(..., number=1) bằng min(timeit.repeat(..., number=1, repeat=15)). Giá trị nhỏ nhất ít bị nhiễu nhất vì nhiễu chỉ có thể làm thời gian tăng lên chứ không làm giảm; đây cũng là cách được khuyến nghị trong tài liệu của module timeit. Mã đo gốc được giữ nguyên; phần cải tiến được đặt thành mục riêng "Cải tiến phép đo" ở cuối notebook n_queens_fast_conflict_calculation_with_numba.ipynb và đo lại trên đúng các bàn cờ như trên. Kết quả ở Bảng 7b.

Kích thước n	Naive O(n²) (µs)	Tuyến tính O(n) (µs)	JIT Numba (µs)	Tỉ lệ naive / tuyến tính
8	47,7	8,7	0,43	5,5
50	662,5	51,4	1,15	12,9
100	2114,3	101,2	1,83	20,9
150	4474,2	165,3	3,90	27,1
200	7844,9	215,8	3,19	36,4
250	11953,7	264,6	3,86	45,2
300	17961,0	338,2	5,05	53,1
350	25923,5	391,4	5,23	66,2
400	32780,4	455,1	5,91	72,0

Kích thước n	Naive O(n²) (µs)	Tuyến tính O(n) (µs)	JIT Numba (µs)	Tỉ lệ naive / tuyến tính
450	39049,0	496,1	6,59	78,7

Bảng 7b. Thời gian tính hàm mục tiêu khi lặp mỗi phép đo 15 lần và lấy giá trị nhỏ nhất

Kết quả cho thấy: (i) đường thời gian ổn định hơn: số lần thời gian bị giảm khi n tăng giảm từ 1 xuống 0 với naive, từ 1 xuống 0 với cách tuyến tính và từ 2 xuống 1 với JIT; (ii) số đo sát thời gian thực hơn, ví dụ với n = 8 JIT đo một lần ra 2,17 µs nhưng lặp 15 lần chỉ còn 0,43 µs (cách đo một lần bị nhiễu làm chậm khoảng 5 lần); (iii) kết luận không đổi mà chắc chắn hơn: ở n = 8 JIT nhanh hơn cách tuyến tính khoảng 20 lần và nhanh hơn cách ngây thơ khoảng 110 lần, ở n = 450 cách tuyến tính nhanh hơn cách ngây thơ khoảng 79 lần và JIT nhanh hơn cách tuyến tính khoảng 75 lần. JIT vẫn còn một điểm lệch nhỏ (n = 200 thấp hơn n = 150) vì thời gian chỉ vài micro giây; có thể giảm thêm bằng cách tăng repeat, tăng số bàn cờ hoặc dùng lệnh %timeit của Jupyter. Vì vậy, khi đo thời gian các hàm chạy rất nhanh, nên lặp lại nhiều lần và lấy giá trị nhỏ nhất thay vì chỉ đo một lần.

8. Bài 7 – Tìm kiếm cục bộ cho bài toán người du lịch (TSP)

8.1. Mô hình hóa

Thành phần	Mô hình hóa
Trạng thái	Hoán vị π của n thành phố (thứ tự thăm).
Hàm mục tiêu	tour_length(π) = d(π(n), π(1)) + Σ d(π(i), π(i+1)); cần cực tiểu hóa.
Lân cận (toán tử cục bộ)	Hoán đổi hai thành phố ở vị trí i < j; có n(n−1)/2 lân cận. Toán tử này dùng cho **mọi** thuật toán so sánh; riêng mục 8.3 so sánh thêm toán tử 2-opt (đảo ngược đoạn tour giữa i và j).
Dữ liệu	n thành phố sinh ngẫu nhiên trong hình vuông đơn vị; khoảng cách Euclid.

Bảng 8. Mô hình hóa TSP

Các thuật toán cài đặt gồm: Steepest-ascent (duyệt mọi cặp, chọn tour ngắn nhất), Steepest-ascent với 10 lần random restart, Stochastic HC (chọn ngẫu nhiên trong các nước cải thiện), First-choice HC (một lân cận ngẫu nhiên mỗi lần, ngưỡng dừng $x = \lceil N(ln\ N + 2) \rceil$ với $N = n[(n-1)/2]$) và Simulated Annealing (lịch mũ, $T_0 = 0{,}3 \rightarrow 10^{-3}$ sau 3000·n vòng lặp).

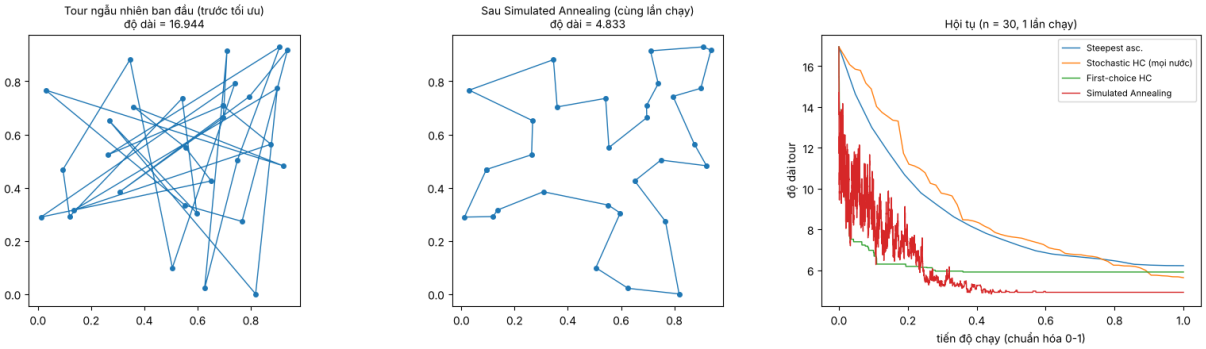
8.2. Kết quả so sánh

Với mỗi n ∈ {10, 20, 30, 50} ta sinh một bài toán TSP cố định rồi chạy mỗi thuật toán nhiều lần (10, 8, 5, 3 lần theo n; 3 lần cho restart khi n ≥ 30) từ các tour ngẫu nhiên khác nhau:

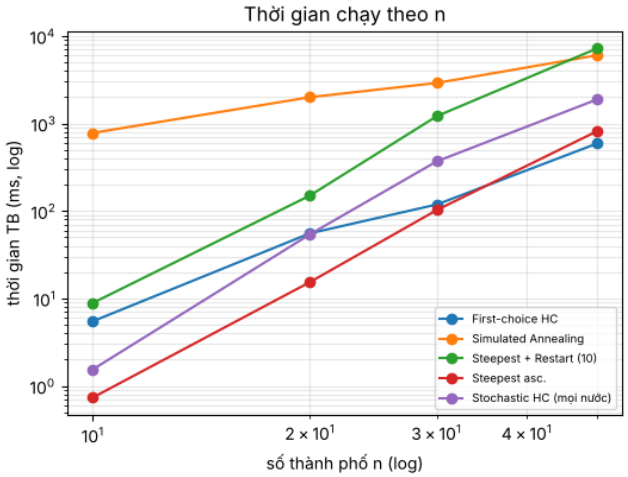
Thuật toán	n	Số lần chạy	Tour tốt nhất	Tour TB	Cải thiện TB (%)	Số lần đánh giá TB	Thời gian TB (ms)
Steepest asc.	10	10	2.964	3.030	36.8	288	0.7
Steepest + Restart (10)	10	10	2.964	2.964	38.1	2907	8.9
Stochastic HC (mọi nước)	10	10	2.964	3.093	35.7	585	1.5
First-choice HC	10	10	2.964	2.964	38.1	377	5.5
Simulated Annealing	10	10	2.964	2.964	38.1	30000	780.4
Steepest asc.	20	8	4.586	4.765	50.2	2613	15.4
Steepest + Restart (10)	20	8	4.148	4.254	55.5	25864	150.0
Stochastic HC (mọi nước)	20	8	4.206	4.535	52.6	6484	54.0
First-choice HC	20	8	4.199	4.522	52.7	2297	55.5
Simulated Annealing	20	8	4.089	4.186	56.2	60000	2000.0
Steepest asc.	30	5	5.922	6.179	60.0	10875	102.6
Steepest + Restart (10)	30	3	5.198	5.245	67.8	108605	1214.9
Stochastic HC (mọi nước)	30	5	5.441	5.835	62.0	27405	370.2
First-choice HC	30	5	5.466	5.795	62.5	5619	118.7
Simulated Annealing	30	5	4.833	4.993	67.6	90000	2909.2
Steepest asc.	50	3	7.870	8.131	67.7	66150	816.4
Steepest + Restart (10)	50	3	7.266	7.403	70.6	578200	7208.3
Stochastic HC (mọi nước)	50	3	8.216	8.587	65.8	146183	1880.8
First-choice HC	50	3	7.288	7.403	70.6	20323	590.4

Thuật toán	n	Số lần chạy	Tour tốt nhất	Tour TB	Cải thiện TB (%)	Số lần đánh giá TB	Thời gian TB (ms)
Simulated Annealing	50	3	6.011	6.310	74.9	150000	6021.6

Bảng 9. So sánh các thuật toán trên TSP (độ dài tour càng nhỏ càng tốt)



Hình 6. Tour ngẫu nhiên ban đầu và tour sau SA của cùng một lần chạy, cùng đường hội tụ (n = 30)



Hình 7. Thời gian chạy theo số thành phố (log-log)

Nhận xét. Các thuật toán đều cải thiện vượt trội so với tour ngẫu nhiên (giảm hàng chục %). Ở n = 10, nhiều khả năng là tối ưu (2,964). Tuy nhiên ở n = 50, chất lượng phân hóa rõ: SA đạt kết quả tốt nhất (trung bình 6,31), kế đến là First-choice và Steepest + Restart (~7,40), kém nhất là Steepest và Stochastic (8,13–8,59). Về độ phức tạp, hai thuật toán duyệt toàn bộ lân cận tăng thời gian nhanh nhất do tốn $O(n^2)$ đánh giá mỗi bước; First-choice rẻ hơn nhưng vướng ngưỡng dừng; SA chậm ở n nhỏ nhưng tốc độ tăng theo n là chậm nhất (3000n vòng lặp). Hình 6 minh họa hiệu quả của SA khi giảm độ dài tour từ 16,94 xuống 4,83.

8.3. Toán tử hoán đổi so với 2-opt (n = 30)

Toán tử	Thuật toán	Tour tốt nhất	Tour TB	Thời gian TB (s)
Hoán đổi (swap)	First-choice HC	5.466	5.795	0.173
Hoán đổi (swap)	Simulated Annealing	4.833	4.993	2.905
2-opt	First-choice HC	4.833	4.942	0.144
2-opt	Simulated Annealing	4.825	4.841	3.688

Bảng 10. Ảnh hưởng của toán tử lân cận tới chất lượng lời giải (5 lần chạy)

Do chỉ thay đổi 2 cạnh (thay vì tối đa 4 cạnh như phép hoán đổi), toán tử 2-opt giúp không gian tìm kiếm mượt hơn và hạn chế cực trị cục bộ giả. Nhờ đó, 2-opt mang lại hiệu quả vượt trội: First-choice đạt tour trung bình 4,94 so với 5,80 của hoán đổi (thời gian tương đương 0,14–0,17 s), trong khi SA kết hợp 2-opt đạt kết quả tốt và ổn định nhất (trung bình 4,84). Chi phí tính toán mỗi bước của hai toán tử là tương đương do có cùng kích thước lân cận n(n-1)/2.

8.4. Thuật toán di truyền (mở rộng)

Ta cài đặt thêm thuật toán di truyền (GA) với quần thể 100 tour, chọn lọc giải đấu (k = 3), lai ghép thứ tự OX, đột biến đảo đoạn (xác suất 0,2), giữ lại 2 cá thể tốt nhất và 300 thế hệ. Trên bài toán n = 30, GA cho tour dài 5,14 (2,5 s), kém SA một chút (Bảng 9: SA tốt nhất 4,83, trung bình 4,99) nhưng tốt hơn các thuật toán leo đồi một lần chạy, với số lần đánh giá cùng bậc (30 000 so với 90 000 vòng lặp của SA); GA nhạy với kích thước quần thể và tỉ lệ đột biến.

9. Bài 8 – Tổng hợp thí nghiệm và phân tích

9.1. Bảng kết quả tổng hợp

Mỗi dòng n quân hậu là trung bình của **100 lần chạy** từ các bàn cờ ngẫu nhiên khác nhau (seed cố định); các dòng TSP lấy từ Bảng 9 (n = 20). Với n quân hậu, "số bước" là số nước đi được thực hiện (với SA là số nước đi được chấp nhận) và "số lần đánh giá" là số lần gọi hàm conflicts(); thành công nghĩa là kết thúc với f = 0.

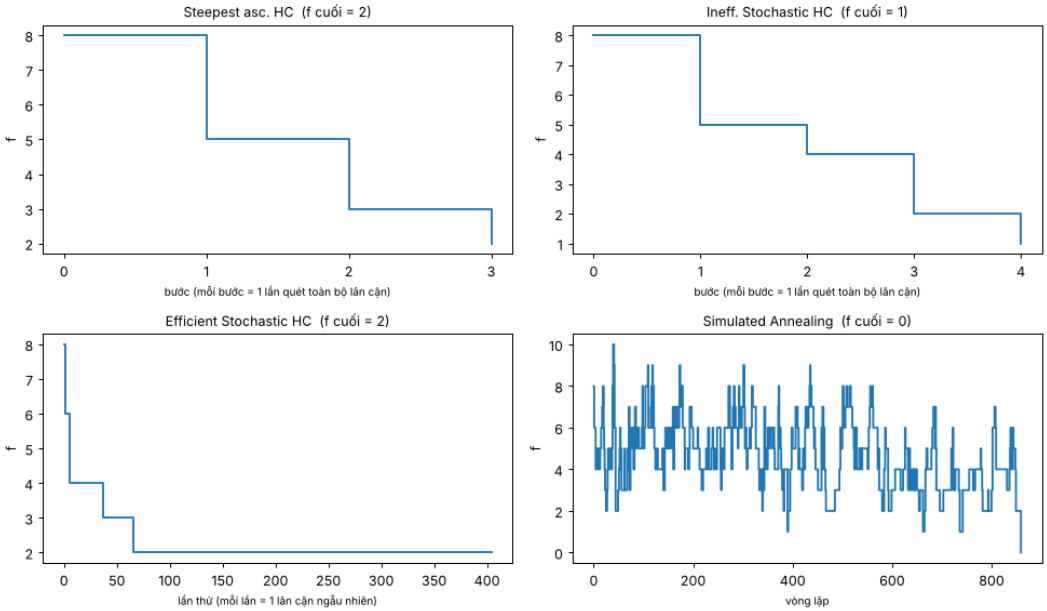
Bài toán	Thuật toán / cấu hình	Kích thước	f tốt nhất	f TB	Tỉ lệ thành công	Số bước TB	Số lần đánh giá TB	Thời gian TB (ms)
n-Queens	Steepest asc. HC	4	0	0.57	47%	1.5	24	0.10
n-Queens	Ineff. Stochastic HC	4	0	0.89	29%	1.8	30	0.12
n-Queens	Efficient Stochastic HC	4	0	0.70	36%	1.8	46	0.40

Bài toán	Thuật toán / cấu hình	Kích thước	f tốt nhất	f TB	Tỉ lệ thành công	Số bước TB	Số lần đánh giá TB	Thời gian TB (ms)
n-Queens	Simulated Annealing	4	0	0.00	100%	72.5	98	0.99
n-Queens	Steepest asc. + Restart	4	0	0.00	100%	4.1	67	0.32
n-Queens	Steepest asc. HC	8	0	1.20	19%	3.3	232	1.64
n-Queens	Ineff. Stochastic HC	8	0	1.34	11%	5.1	334	1.91
n-Queens	Efficient Stochastic HC	8	0	1.44	8%	4.8	374	4.49
n-Queens	Simulated Annealing	8	0	0.01	99%	655.9	1641	28.78
n-Queens	Steepest asc. + Restart	8	0	0.00	100%	24.7	1763	11.55
TSP	Steepest asc.	20	4.586	4.765	—	—	2613	15.4
TSP	Steepest + Restart (10)	20	4.148	4.254	—	—	25864	150.0
TSP	Simulated Annealing	20	4.089	4.186	—	—	60000	2000.0

Bảng 11. Bảng tổng hợp thí nghiệm (n quân hậu: 100 lần chạy; TSP: 8 lần chạy với n = 20, f là độ dài tour)

Nhận xét. Ở n = 8, leo đồi một lần chạy chỉ giải được 8–19% do 81–92% lượt chạy bị kẹt ở cực trị cục bộ. SA giải quyết triệt để với tỷ lệ thành công 99% (100% ở n = 4), nhưng đổi lại chi phí cao (~29 ms, ~1640 lần đánh giá). Với quy mô nhỏ này, Steepest-ascent kết hợp Random Restart là lựa chọn tối ưu hơn khi đạt 100% thành công chỉ với ~11,6 ms. Ở n = 4, không gian chỉ gồm 256 trạng thái nên tốc độ các thuật toán đều rất nhanh, sự khác biệt nằm ở tỷ lệ thành công. Kết quả trên TSP đồng nhất với bài toán n quân hậu: Restart và SA giúp cải thiện rõ rệt các tour kém nhất của leo đồi đơn.

9.2. Hành vi hội tụ

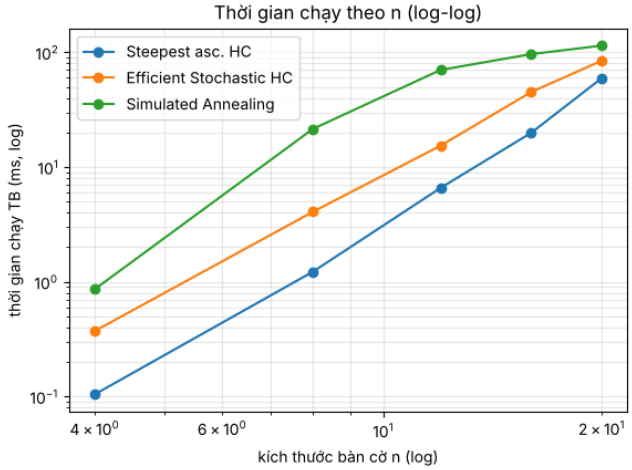


Hình 8. Đường hội tụ của một lần chạy đại diện cho mỗi thuật toán trên bài toán 8 quân hậu

Hai thuật toán Steepest-ascent và Stochastic HC giảm f nhanh lúc đầu nhưng sớm dừng ở dạng phẳng ($f > 0$) do kẹt tại cực trị cục bộ (tỷ lệ kẹt ở $n = 8$ là 81% và 89%). First-choice xuất hiện các bậc thang dài do phần lớn nước đi bị từ chối, chỉ dừng sau x lần thử thất bại và dễ kẹt ở plateau do chỉ chấp nhận cải thiện ngặt (tỷ lệ kẹt 92%). Trái lại, Simulated Annealing dao động mạnh ở nhiệt độ cao và dừng khi đạt $f = 0$. SA vượt trội hoàn toàn khi chỉ có 1% lượt chạy kẹt ở cực trị cục bộ, trong khi ba thuật toán leo đồi còn lại bị kẹt tới 81–92%.

9.3. Khả năng mở rộng theo kích thước bàn cờ

Với $n = 4, 8, 12, 16, 20$ và 30 lần chạy mỗi kích thước, ta đo thời gian trung bình của một lần chạy cho ba thuật toán (Hình 9, thang log-log) và ước lượng số mũ k trong $t \sim n^k$ bằng độ dốc của đường hồi quy log t theo log n. Vì k được ước lượng từ 5 điểm đo và thời gian đo có nhiễu, k có thể dao động khoảng ±0,3 giữa các lần chạy nên chỉ nên đọc như xu hướng.



Hình 9. Thời gian chạy theo n (log-log)

Thuật toán	t(n=4) ms	t(n=8) ms	t(n=12) ms	t(n=16) ms	t(n=20) ms	k (mọi n)	k (n ≥ 12)
Steepest asc. HC	0.1	1.2	6.6	19.9	59.3	3.91	4.28
Efficient Stochastic HC	0.4	4.1	15.3	45.1	84.2	3.40	3.35
Simulated Annealing	0.9	21.5	70.2	96.3	114.2	3.08	0.96

Bảng 12. Thời gian chạy trung bình (ms) và số mũ thực nghiệm k

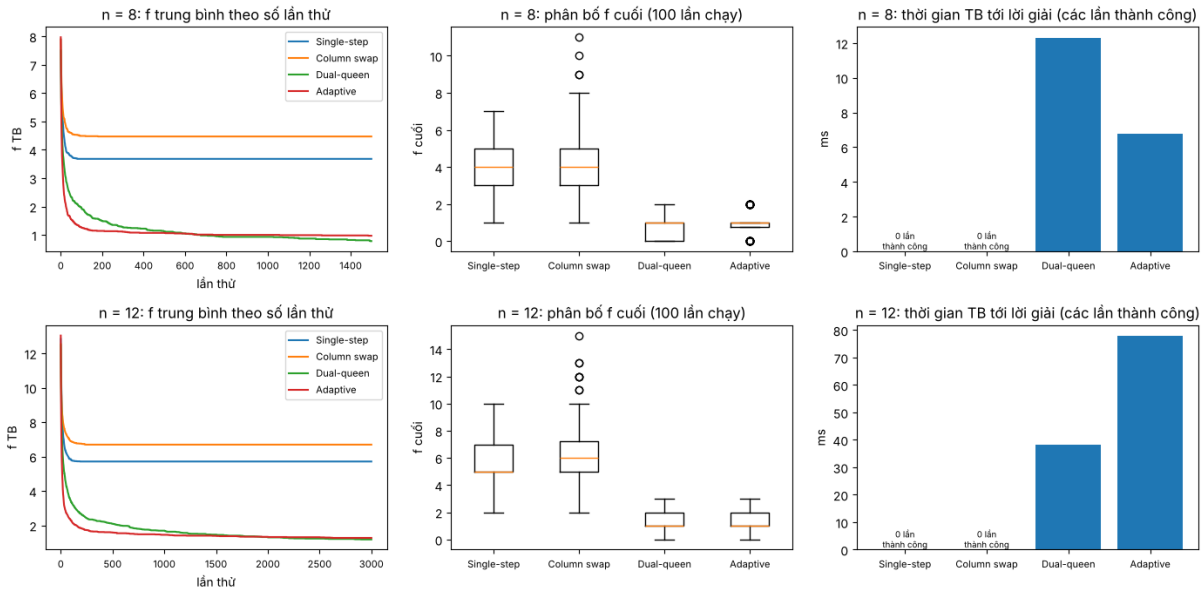
Thuật toán	% thành công n=4	% thành công n=8	% thành công n=12	% thành công n=16	% thành công n=20
Steepest asc. HC	43%	3%	7%	3%	3%
Efficient Stochastic HC	37%	7%	7%	0%	0%
Simulated Annealing	100%	100%	83%	70%	37%

Bảng 13. Tỉ lệ thành công theo kích thước (30 lần chạy mỗi ô, một lần chạy đơn, không restart)

Về độ phức tạp thời gian, Steepest-ascent có tốc độ tăng k ≈ 3,9 (4,3 khi n ≥ 12) do tốn O($\boldsymbol{n^3}$) mỗi bước [n(n-1) lân cận x O(n)], còn First-choice đạt k ≈ 3,4 (thử O(n) với ngưỡng dừng x ≈ $\boldsymbol{n^2}$ ln(n)). SA mở rộng thời gian tốt nhất: nhờ ngân sách 5000 vòng lặp (O(n)/vòng), khi n ≥ 12 thời gian tăng gần tuyến tính (k ≈ 1,0), còn tính chung mọi n đạt k ≈ 3,1 (do dừng sớm khi n nhỏ). Tuy nhiên, ngân sách cố định làm tỷ lệ thành công của SA giảm từ 100% (n = 8) xuống 37% (n = 20), còn leo đồi gần như kẹt hoàn toàn ở n ≥ 8 (chỉ đạt 3–7%, thậm chí 0%). Như vậy, leo đồi "nhanh" nhưng cho kết quả sai; muốn SA giải bàn cờ lớn cần tăng số vòng lặp tối đa theo n

9.4. Các toán tử di chuyển khác (phần nâng cao)

Ta cài đặt bốn toán tử và chạy First-choice HC với mỗi toán tử 100 lần trên 8 quân hậu (ngân sách 1500 lần thử) và 12 quân hậu (3000 lần thử): (1) *single-step*: dịch một quân hậu lên/xuống đúng một ô; (2) *column swap*: hoán đổi hàng của hai quân hậu ở hai cột ngẫu nhiên; (3) *dual-queen*: chuyển hai quân hậu cùng lúc tới hàng ngẫu nhiên; (4) *adaptive*: với xác suất 0,7 di chuyển quân hậu đang bị xung đột nhiều nhất sang hàng ngẫu nhiên, ngược lại chọn ngẫu nhiên một trong ba toán tử còn lại.



Hình 10. Chất lượng trung bình theo số lần thử, phân bố f cuối và thời gian tới lời giải của bốn toán tử (hàng trên n = 8, hàng dưới n = 12)

n	Toán tử	% thành công	TB f cuối	TB thời gian thành công (ms)
8	Single-step	0	3,68	—
8	Column swap	0	4,47	—
8	Dual-queen	30	0,78	12,3
8	Adaptive	25	0,97	6,8
12	Single-step	0	5,71	—
12	Column swap	0	6,69	—
12	Dual-queen	12	1,18	38,4
12	Adaptive	10	1,27	77,8

Bảng 14. So sánh các toán tử di chuyển (100 lần chạy mỗi dòng)

Single-step và *column swap* không giải được lần nào. Single-step chỉ nhìn một lân cận rất nhỏ nên kẹt rất sớm. Column swap có lý do cấu trúc: hoán đổi hàng giữ nguyên *tập* các hàng đang dùng, nên nếu bàn cờ ban đầu (ngẫu nhiên) có hai quân cùng hàng thì xung đột hàng không bao giờ mất; toán tử này chỉ phù hợp khi khởi tạo từ một hoán vị. *Dual-queen* tốt nhất ở cả hai cỡ: nước đi lớn hơn giúp vượt một số cực trị cục bộ. *Adaptive* cho chất lượng gần dual-queen nhưng không vượt qua nó. Về thời gian, với n = 8 các lần chạy thành công của adaptive còn nhanh hơn dual-queen (6,8 so với 12,3 ms) vì nhắm đúng quân hậu đang gây xung đột; nhưng với n = 12 nó chậm hơn khoảng gấp đôi (77,8 so với 38,4 ms) vì mỗi lần thử phải tính xung đột của từng quân ($O(n^2)$), chi phí này tăng nhanh theo n. Bài học: lân cận càng rộng/đa dạng thì càng dễ thoát cực trị cục bộ nhưng mỗi ứng viên càng ít khả năng tốt hơn, và toán tử phải phù hợp với biểu diễn trạng thái.

10. Nhận xét tổng hợp và Đánh giá

10.1. Hiện tượng bế tắc tại cực trị cục bộ (*Local Optimum*)

Bản chất đồng nhất qua các bài toán: Trải qua thực nghiệm trên cả ba mô hình (mê cung, n quân hậu, và bài toán người du lịch - TSP), thuật toán leo đồi (*Hill Climbing*) đều bộc lộ chung một hạn chế: do vận hành theo cơ chế tham lam hướng tới sự cải thiện tức thời, thuật toán sẽ tự động dừng lại ngay khi cạn kiệt các trạng thái lân cận tốt hơn.

Biểu hiện cụ thể:

- Trong bài toán mê cung, thuật toán kẹt lại tại ô $(5,9)$ cách đích 6 ô do vấp phải tường chắn.
- Với bài toán 8 quân hậu, có tới $81\% - 92\%$ các lượt chạy kết thúc với giá trị xung đột $f > 0$.
- Đối với TSP, các lượt chạy *Steepest-ascent* xuất phát từ những tour ban đầu khác nhau sẽ chốt lại ở các chiều dài tour khác nhau (dao động trong khoảng $4{,}38 - 4{,}95$ ở quy mô $n = 20$).

Kết luận kỹ thuật: Việc thay đổi thứ tự xét hướng duyệt hoặc áp dụng chính sách chọn ngẫu nhiên khi xảy ra đồng hạng (*tie-breaking*) chỉ làm thay đổi quỹ đạo đường đi chứ không giải quyết được điểm dừng bế tắc.

10.2. Hiệu năng của chiến lược khởi động lại ngẫu nhiên (*Random Restart*)

- **Ưu điểm vượt trội:** Thao tác lặp lại quá trình tìm kiếm từ nhiều điểm xuất phát khác nhau đã biến một thuật toán dễ bị mắc kẹt trở thành một chiến lược đạt độ tin cậy gần như tuyệt đối. Cụ thể, bài toán 8 quân hậu đòi hỏi trung bình từ 7 đến 9 lần khởi động để chạm mốc 100% thành công, với chi phí tính toán tăng trưởng gần như tuyến tính theo số lần restart.
- **Hạn chế cốt lõi:** Mỗi lượt khởi động lại đều xóa bỏ toàn bộ tiến độ tích lũy từ các lần chạy trước đó. Thêm vào đó, khi xác suất thành công đơn lẻ suy giảm quá sâu (ở các quy mô n lớn, xác suất thành công chỉ còn $0\% - 7\%$), số lượng lần khởi động yêu cầu sẽ tăng vọt một cách nhanh chóng.

10.3. Đánh giá chiến lược luyện kim mô phỏng (*Simulated Annealing* - SA)

- **Khả năng thoát bẫy tối ưu:** Nhờ cơ chế chấp nhận các bước dịch chuyển tồi hơn dựa trên xác suất $p = \exp\left(-\frac{\Delta}{T}\right)$, thuật toán SA thành công vượt qua các cực trị cục bộ, đạt tỷ lệ 99% thành công trên bàn cờ 8 quân hậu và thiết lập chiều dài tour TSP ngắn nhất (hoặc ngang bằng ở $n = 10$) trên mọi quy mô.
- **Độ nhạy đối với lịch nhiệt độ:** Chất lượng lời giải phụ thuộc quyết định vào hàm giảm nhiệt. Dưới cùng một hạn mức ngân sách, tỷ lệ thành công dao động mạnh từ mức 7% (với lịch mũ $\alpha = 0.9$, làm nguội quá nhanh) lên đến 100% (với lịch *logarithmic*). Quy tắc chung là lịch làm nguội càng chậm thì chất lượng càng cao, nhưng đồng thời tiêu tốn nhiều thời gian tính toán hơn.

10.4. Thời gian thực thi và Khả năng mở rộng quy mô

- **Thuật toán vét cạn toàn bộ lân cận:** Các biến thể *Steepest-ascent* và *Stochastic* có thời gian chạy tăng trưởng theo cấp bậc đa thức bậc cao $n^3 - n^4$.
- **Ưu thế của First-choice:** Biến thể *First-choice* tỏ ra tiết kiệm hơn rõ rệt khi số lượng bước lặp lớn (đặc biệt trong TSP với $n = 50$, chỉ tốn 20.323 lần đánh giá so với 66.150 lần của *Steepest-ascent* và 146.183 lần của *Stochastic*); tuy nhiên, ở bài toán n quân hậu quy mô nhỏ, chi phí sinh lân cận để thỏa mãn ngưỡng dừng lại làm nó trở nên chậm hơn.
- **Hành vi của Simulated Annealing:** SA duy trì thời gian chạy tăng trưởng gần như tuyến tính theo n khi cố định ngân sách vòng lặp, nhưng đòi hỏi phải mở rộng ngân sách tương ứng khi kích thước bàn cờ lớn để đảm bảo tỷ lệ thành công.

10.5. Hạn chế và Khoảng trống của thực nghiệm

- **Quy mô mẫu:** Số lượng lượt chạy thử nghiệm còn khiêm tốn ($30 - 100$ lần với n quân hậu, $3 - 10$ lần với TSP), dẫn đến sai số lấy mẫu dao động trong biên độ vài phần trăm.
- **Tính phụ thuộc môi trường:** Thời gian thực thi đo đạc phụ thuộc trực tiếp vào cấu hình phần cứng thiết bị; mỗi quy mô TSP chỉ sử dụng một bài toán cố định; số liệu thời gian (bao gồm cả JIT Numba) được lấy từ một lần chạy đơn độc trên một máy cố định.
- **Tinh chỉnh tham số:** Các siêu tham số của SA (như nhiệt độ khởi tạo T_0, lịch trình nhiệt, tổng số vòng lặp) được xác định thông qua phương pháp khảo sát thử nghiệm thủ công thay vì áp dụng các thuật toán tối ưu hóa hệ thống tự động.

11. Câu hỏi thảo luận:

Câu 1. Tìm kiếm cục bộ khác tìm kiếm theo đường đi (BFS, A*) ở mục tiêu và dữ liệu cần lưu trữ như thế nào?

Mục tiêu và không gian lưu trữ: BFS và A* có mục tiêu tìm kiếm đường đi hoàn chỉnh từ trạng thái bắt đầu đến đích, do đó bắt buộc phải duy trì cây hoặc đồ thị tìm kiếm (bao gồm hàng đợi biên, tập trạng thái đã thăm, và con trỏ truy hồi nút cha), khiến dung lượng bộ nhớ có thể phình to theo cấp số mũ. Ngược lại, tìm kiếm cục bộ chỉ quan tâm đến việc tìm kiếm trạng thái đích hoặc cấu hình tối ưu của một hàm mục tiêu, không có nhu cầu lưu vết đường đi, vì vậy chỉ cần lưu trữ trạng thái hiện tại (cùng trạng thái tốt nhất ghi nhận được) kèm theo giá trị mục tiêu, giúp bộ nhớ duy trì ổn định ở mức gần như hằng số.

Đặc tính kỹ thuật: BFS và A* đảm bảo tính đầy đủ và tính tối ưu (với A* sử dụng hàm heuristic admissible). Trái lại, tìm kiếm cục bộ không đảm bảo tính đầy đủ cũng như tính tối ưu, và rất dễ rơi vào bẫy cực trị cục bộ.

Câu 2. Vì sao Simple Hill Climbing có thể mắc kẹt ở local optimum, plateau hoặc ridge?

Thuật toán leo đồi đơn giản vận hành dựa trên việc quan sát các láng giềng trực tiếp và chỉ chấp nhận dịch chuyển khi giá trị mục tiêu được cải thiện ngặt. Do tầm nhìn cục bộ đó, ba hiện tượng bế tắc thường xuất hiện:

- **Local optimum (Cực trị cục bộ):** Trạng thái mang lại giá trị tốt hơn mọi láng giềng xung quanh nhưng vẫn chưa phải là cực trị toàn cục (ví dụ như ô $(5,9)$ trong mê cung hay cấu hình n quân hậu dừng lại với $f = 1$).
- **Plateau (Vùng phẳng):** Không gian mà các trạng thái láng giềng đều sở hữu cùng một mức giá trị mục tiêu, làm triệt tiêu hoàn toàn hướng cải thiện (như biến thể *first-choice* tự động dừng khi mọi lần thử nghiệm đều không tạo ra bước tiến tốt hơn ngặt).
- **Ridge (Sống núi):** Chuỗi các cực trị cục bộ nằm lệch hướng so với tập hợp các hành động di chuyển cho phép, khiến mọi bước dịch chuyển đơn lẻ đều làm suy giảm chất lượng, mặc dù việc di chuyển tuần tự dọc theo sống núi sẽ dẫn tới đỉnh tối ưu.

Câu 3. So sánh Steepest-ascent, Stochastic và First-choice về số lần đánh giá hàm mục tiêu

Chi phí tính toán mỗi bước lặp: Hai biến thể *Steepest-ascent* và *Stochastic* (xét mọi nước đi) luôn bắt buộc phải duyệt và đánh giá toàn bộ $n(n-1)$ trạng thái lân cận (tương đương 56 lần với bàn cờ $n = 8$). Trong khi đó, biến thể *First-choice* chỉ sinh ngẫu nhiên và đánh giá duy nhất một lân cận cho mỗi lần thử, nhưng nếu không cải thiện thì phải thử liên tục đến ngưỡng $x \approx N(\ln N + 2)$ lần mới dừng lại.

Thực nghiệm trên 8 quân hậu: Tổng số lần đánh giá trung bình trong toàn bộ tiến trình là 232 lần đối với *Steepest-ascent*, 334 lần đối với *Stochastic*, và 374 lần đối vơi *First-choice*. Do số lượng bước dịch chuyển thực tế rất ngắn (chỉ khoảng 3–5 bước), chi phí tích lũy của *First-choice* trở nên cao hơn.

Hiệu năng trên quy mô lớn (TSP với $n = 50$): Khi bài toán đòi hỏi nhiều bước lặp hơn, *First-choice* thể hiện ưu thế tiết kiệm vượt trội với chỉ 20.323 lần đánh giá, thấp hơn rất nhiều so với 66.150 lần của *Steepest-ascent* và 146.183 lần của *Stochastic*. Về chất lượng nghiệm trên 8 quân hậu, tỷ lệ thành công tương ứng lần lượt là 19% (*Steepest-ascent*), 11% (*Stochastic*) và 8% (*First-choice*).

Câu 4. Random restart cải thiện xác suất tìm lời giải như thế nào? Khi nào tăng số lần restart không còn hiệu quả?

Cơ chế xác suất: Giả sử một lần thực thi đơn đạt được thành công với xác suất p, thì sau k lần khởi động lại độc lập, xác suất xuất hiện ít nhất một lần thành công sẽ đạt $1 - (1-p)^k$, với kỳ vọng số lần khởi động cần thiết xấp xỉ $\frac{1}{p}$.

Đánh giá thực nghiệm: Với bài toán 8 quân hậu ($p \approx 8\% - 19\%$), thuật toán cần trung bình từ 7 đến 9 lần khởi động và đạt 100% thành công dưới giới hạn 100 lần thử. **Giới hạn hiệu quả:** Việc gia tăng số lần restart trở nên lãng phí trong hai trường hợp:

1. Khi xác suất p đã đủ lớn (kết quả không thể cải thiện thêm mà chỉ làm tăng chi phí tính toán).

2. Khi xác suất p quá nhỏ (ở quy mô n lớn, $p \approx 0\% - 7\%$, đòi hỏi hàng chục lần restart trong khi mỗi lần tốn kém hàng chục mili-giây, vì bản chất mỗi lần khởi động hoàn toàn xóa bỏ mọi thông tin tích lũy từ các lần trước đó).

Câu 5. Vì sao SA có thể chấp nhận nghiệm xấu hơn? Nhiệt độ ảnh hưởng thế nào đến khám phá và khai thác?

Bản chất chấp nhận bước xấu: Thông qua xác suất chuyển trạng thái $p = \exp\left(-\frac{\Delta}{T}\right)$, thuật toán Luyện kim mô phỏng (*Simulated Annealing*) có khả năng vượt qua các bẫy cực trị cục bộ để di chuyển sang các thung lũng nghiệm sâu hơn.

Vai trò điều phối của nhiệt độ T:

- *Nhiệt độ cao:* Xác suất chấp nhận bước xấu lớn, giúp thuật toán thực hiện quá trình **khám phá diện rộng** ($exploration$) không gian trạng thái, hoạt động gần giống như bước đi ngẫu nhiên.
- *Nhiệt độ thấp:* Xác suất chấp nhận bước xấu thu hẹp dần về 0, ép thuật toán chuyển sang chế độ **khai thác sâu** ($exploitation$), hành xử tương tự như thuật toán *Hill Climbing* truyền thống quanh vùng lân cận.

Chứng minh thực nghiệm: Quá trình làm nguội quá nhanh ($\alpha = 0.9$) chỉ mang lại 7% tỷ lệ thành công, trong khi chiến lược làm nguội chậm ($\alpha = 0.999$) đạt tới 98% thành công**.**

Câu 6. Các lịch tuyến tính, hàm mũ, fast và logarithmic khác nhau ra sao? Lịch nào phù hợp trong thí nghiệm?

- **Đặc trưng các mô hình lịch nhiệt:**
  - *Lịch tuyến tính (Linear):* Suy giảm đều đặn và về mức 0 sau tổng số vòng lặp T_0/η.
  - *Lịch mũ (Exponential):* Giảm theo tỷ lệ cố định mỗi vòng (tốc độ nhanh ở giai đoạn đầu, chậm dần về sau).
  - *Lịch nhanh (Fast):* Định dạng $\frac{T_0}{1+t}$, giảm cực nhanh ở giai đoạn đầu rồi duy trì một đuôi giá trị rất thấp.
  - *Lịch logarit (Logarithmic):* Định dạng $\frac{T_0}{\ln(1+t)}$, giảm nhanh lúc đầu nhưng sau đó đi ngang rất lâu, duy trì xác suất chấp nhận nhỏ khác 0 trong khoảng thời gian dài (đáp ứng đúng cam kết lý thuyết về tối ưu toàn cục nhưng tốc độ rất chậm).
- **Đánh giá thực nghiệm với 8 quân hậu ($T_0 = 2$):** Lịch *logarithmic* đạt 100% thành công với khoảng 712 vòng lặp; lịch mũ $\alpha = 0.999$ đạt 98% với khoảng 1.717 vòng; lịch *fast* đạt 88% với 733 vòng; lịch tuyến tính $\eta = 0.0004$ đạt 93% với 3.296 vòng. Do đó, lịch *logarithmic* và lịch mũ chậm là các phương án tối ưu nhất, trong khi lịch tuyến tính đòi hỏi phải tinh chỉnh hệ số η rất nhỏ.

Câu 7. Vì sao biểu diễn n quân hậu mỗi cột đúng một quân giúp giảm không gian tìm kiếm?

Thu gọn quy mô không gian: Nếu cho phép đặt tự do n quân hậu trên bàn cờ $n \times n$, tổng số cách sắp xếp là $C(n^2, n)$ (đạt tới 4.426.165.368 cách với $n = 8$).

Tối ưu hóa bằng ràng buộc: Việc áp dụng ràng buộc mỗi cột đặt đúng một quân quy đổi trạng thái thành một véc-tơ gồm n giá trị hàng, thu hẹp không gian xuống còn n^n trạng thái ($8^8 = 16.777.216$ với $n = 8$). Cấu trúc này triệt tiêu hoàn toàn hiện tượng xung đột cột (không cần kiểm tra) và cho phép định nghĩa thao tác lân cận cực kỳ đơn giản (thay đổi hàng trong một cột với $n(n-1)$ lân cận). Nếu bổ sung thêm ràng buộc mỗi hàng đúng một quân (tạo thành cấu trúc hoán vị), không gian tìm kiếm thu gọn tối đa chỉ còn $n! = 40.320$ trạng thái.

Câu 8. Vì sao đếm xung đột theo hàng/đường chéo có thể có độ phức tạp $O(n)$ thay vì $O(n^2)$?

Bản chất toán học: Hai quân hậu tấn công lẫn nhau khi chúng cùng nằm trên một hàng hoặc cùng một đường chéo. Nếu có tổng cộng k quân hậu nằm trên cùng một hàng hoặc đường chéo, chúng sẽ tạo ra chính xác $C(k,2) = \frac{k(k-1)}{2}$ cặp xung đột.

Thuật toán tuyến tính: Thay vì thực hiện so sánh từng cặp đôi quân hậu theo kiểu ngây thơ với độ phức tạp $C(n,2) = O(n^2)$, thuật toán chỉ cần một vòng lặp duyệt qua n quân hậu để cập nhật giá trị vào ba mảng đếm (hàng, đường chéo chính $i + q_i$, đường chéo phụ $i - q_i + n$), sau đó cộng dồn giá trị $C(k,2)$ từ các ô đếm: tổng độ phức tạp đạt mức $O(n)$.

Câu 9. Khi dùng Numba, vì sao phải loại trừ thời gian biên dịch lần đầu trong phép đo tốc độ?

Cơ chế hoạt động của JIT: Hàm được trang trí bằng @njit chỉ tiến hành biên dịch mã nguồn thành mã máy native tại **lần gọi thực thi đầu tiên**. Lượt gọi này gộp chung cả khoảng thời gian biên dịch (kéo dài tới hàng trăm mili-giây), lớn hơn rất nhiều so với thời gian chạy thực tế của hàm (chỉ tính bằng vài micro-giây).

Hệ quả thực nghiệm: Nếu không tách bỏ thời gian biên dịch này, kết quả đo đạc sẽ phản ánh sai lệch rằng JIT chậm hơn cả cách viết Python thuần túy. Do đó, bắt buộc phải gọi hàm trước một lần để hoàn tất quá trình biên dịch, sau đó mới tiến hành đo lường trên nhiều bàn cờ để tính giá trị trung bình.

Câu 10. Với TSP, thay toán tử swap bằng 2-opt có thể ảnh hưởng đến chất lượng lời giải và số láng giềng như thế nào?

Đặc tính hình học của toán tử: Hoán đổi vị trí hai thành phố i và j (swap) có thể làm thay đổi tới 4 cạnh của chu trình, khiến một thay đổi nhỏ về cấu trúc biểu diễn gây ra biến động lớn về chiều dài tour. Ngược lại, toán tử **2-opt** (đảo ngược thứ tự đoạn đường đi giữa i và j) chỉ thay đổi đúng 2 cạnh và trực tiếp loại bỏ các cạnh giao cắt nhau, giúp không gian tìm kiếm trở nên mượt mà (trơn tru hơn) và hạn chế tối đa các cực trị cục bộ giả.

Quy mô lân cận và thực nghiệm: Số lượng trạng thái lân cận của cả hai toán tử đều duy trì ở mức $\frac{n(n-1)}{2}$. Kết quả thực nghiệm với $n = 30$ (*Bảng 10*) minh chứng rằng phương pháp *First-choice* kết hợp toán tử 2-opt đạt chiều dài tour trung bình 4.94 (so với 5.80 khi dùng swap), trong khi *Simulated Annealing* kết hợp 2-opt mang lại chất lượng lời giải tối ưu và ổn định nhất (trung bình 4.84; mức tốt nhất đạt 4.83).

12. Kết luận và Hướng phát triển

12.1. Đánh giá tổng quan về thuật toán Tìm kiếm cục bộ

Đặc tính cốt lõi: Các kết quả thực nghiệm minh chứng rằng tìm kiếm cục bộ là một họ thuật toán đạt hiệu quả vượt trội về mặt dung lượng bộ nhớ và tốc độ thực thi, tuy nhiên mang bản chất **không đầy đủ** (*incomplete*).

So sánh các chiến lược giải quyết bế tắc:

- **Hill Climbing thuần túy:** Thường xuyên sa lầy tại các điểm cực trị cục bộ (ghi nhận từ $81\% - 92\%$ số lần chạy trên bàn cờ 8 quân hậu đều kết thúc với giá trị xung đột $f > 0$).
- **Random Restart:** Khắc phục hiệu quả tình trạng kẹt nghiệm bằng cơ chế lặp lại quá trình khởi tạo từ nhiều điểm xuất phát khác nhau.
- **Simulated Annealing (SA):** Giải pháp tối ưu hóa toàn diện thông qua cơ chế chấp nhận có kiểm soát các bước dịch chuyển kém tối ưu hơn, đạt tỷ lệ thành công cao nhất ($99\% - 100\%$) với cái giá phải trả là yêu cầu hiệu chỉnh phức tạp đối với lịch trình nhiệt độ.

Yếu tố quyết định chất lượng nghiệm: Hiệu năng của tìm kiếm cục bộ phụ thuộc trực tiếp vào hai yếu tố then chốt:

1. Thiết kế toán tử lân cận (ví dụ: cấu trúc *dual-queen* tối ưu hơn *single-step*; toán tử *2-opt* vượt trội hơn phép hoán đổi *swap*).
2. Lựa chọn chiến lược lịch giảm nhiệt độ phù hợp.

12.2. Hướng phát triển

Nhằm khắc phục các hạn chế hiện tại và nâng cao hiệu suất của hệ thống, các hướng nghiên cứu và cải tiến tiếp theo bao gồm:

- **Tối ưu hóa chi phí tính toán:** Cài đặt cơ chế tính toán gia tăng (*delta evaluation*) cho hàm mục tiêu nhằm giảm thiểu tối đa chi phí xử lý trong mỗi bước lặp.
- **Hệ thống hóa tham số:** Xây dựng phương pháp hiệu chỉnh các tham số của thuật toán Simulated Annealing một cách tự động và bài bản thay vì khảo sát thủ công.
- **Độ tin cậy thực nghiệm:** Lặp lại các phép đo hiệu năng (bao gồm cả phiên bản tăng tốc bằng Numba JIT) với số lượng mẫu lớn hơn nhằm triệt tiêu tối đa nhiễu đo đạc.
- **Mở rộng mô hình:** Thử nghiệm các phương pháp tìm kiếm dựa trên quần thể, tiêu biểu như Thuật toán di truyền (*Genetic Algorithm*), áp dụng cho bài toán n quân hậu và các bài toán tối ưu quy mô lớn khác.

Bảng phân công:

MSSV	Họ Tên	Phân công	Đóng góp
3124720026	Huỳnh Tấn Phi Hùng	Viết báo cáo cho bài 3 ,4 và trả lời các câu hỏi 3, 4, 5. Lập trình Task 1–5 trong n_queens.ipynb	25%
3124720072	Đỗ Xuân Triệu	Viết báo cáo cho bài 7, 8 và trả lời các câu hỏi 7, 10. Lập trình travel-ing_sales-man_prob-lem.ipynb, Task 6 và phần nâng cao trong n_queens.ipynb	25%
3124720050	Lê Tấn Phát	Viết báo cáo cho bài 1, 2 và trả lời các câu hỏi 1 ,2	25%
3124720053	Lê Ngọc Phúc	Viết báo cáo cho bài 5, 6 và trả lời các câu hỏi 6, 8, 9	25%